ER Model

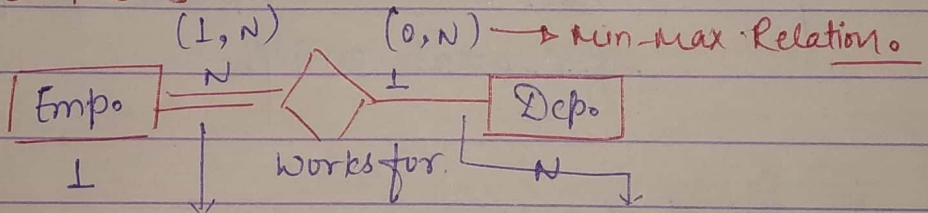
- Entity Type / Relationship type : eg. Person(Age, Name)
- Entity / Relationship : eg. (26, ABC).

→ State of entity : All record in entity.

- Attributes : → Composite Vs Simple
  → Single Vs Multivalued
  → Stored Vs Derived
  - Complex : Composite + Multivalued ⇒ Eg. Address

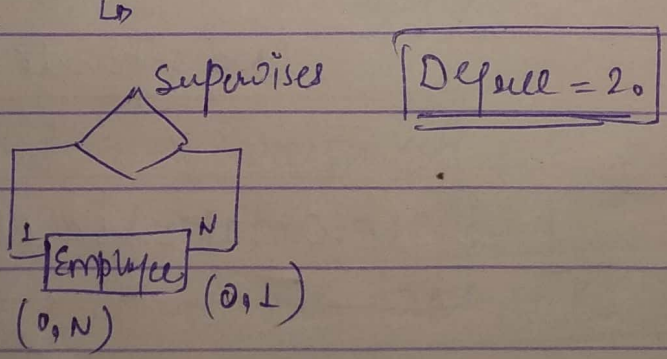
- Degree : Entities participating in a relation.
- Cardinality : # Maximum no of relationships in which an entity can participate
- Participation / Existance : Minimum no of relationship an entity can participate.

→ Structure Constraints



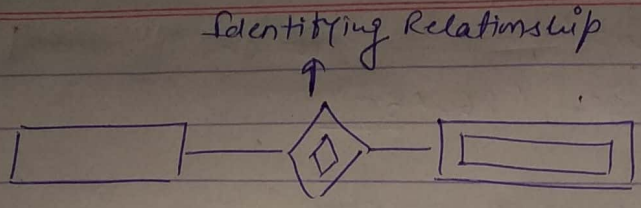
If all entities are participating in a relationship.

- Recursive Relation :



Scanned by CamScanner

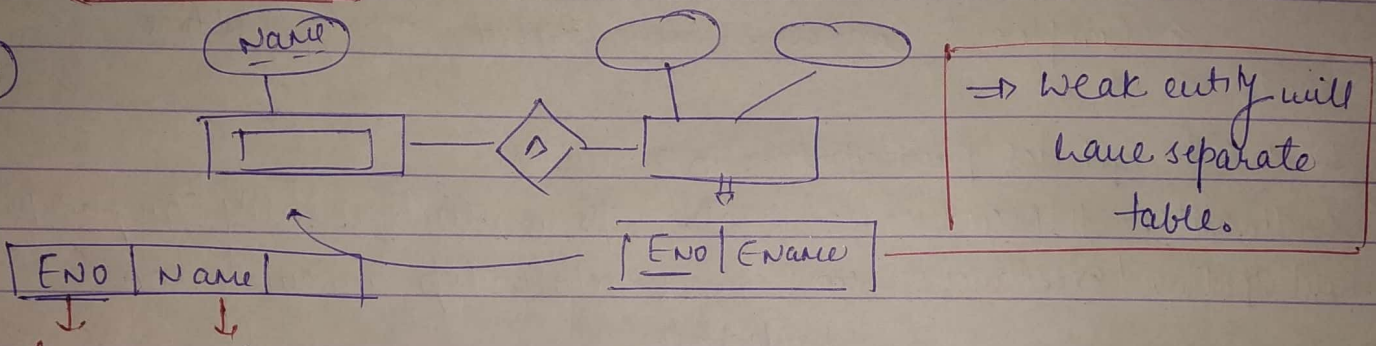
Weak entity:



No primary key. → It Always have total participation.

⇒ ER to Relational Table:

(1)

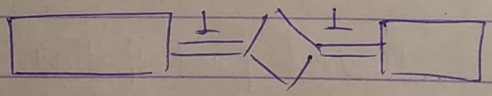


⇒ Weak entity will have separate table.

foreign key partial key

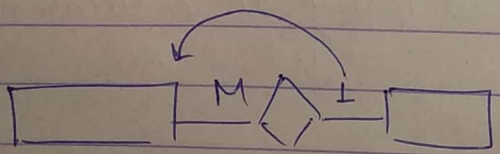
↳ Add all the attribute of primary key for weak entity.

(2)



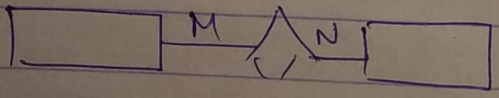
⇒ Can't combine. foreign key at both side. (No separate table)

(3)



⇒ Move toward many side [Also for 1 to many] (No separate table)

(4)



⇒ New table for Relation.

Scanned by CamScanner

ER Model	Relation Model
→ 1:1 or 1:N Relationship	foreign key (or Table / Relation)
→ M:N Relation	Relationship "Relation" + 2 FK.
→ n-ary relationship	Relationship "Relation" + n FK.
→ Composite attribute	set of simple component attribute.
→ Multivalued attribute	Relation + foreign key. (create new table).
→ Degree: # No of entities in a relationship	→ Degree: No of attributes.

Relational Constraints:

① Domain: → Atomic attribute → Composite (make all simple)
→ Multivalued (×) [create new table]

② Key Constraint: → primary (not null) → Unique (null allowed)
→ Super key
→ Candidate key.

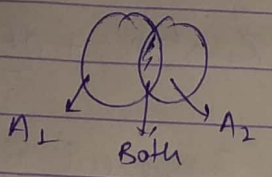
③ Referential: → foreign key → Cascade or set NULL or some other value → on delete opn.
→ Can be NULL. → Can be duplicate.
→ One table [FK] refered to another table

→ Not necessary to be primary key in another table.

Scanned by CamScanner

④ Entity : primary key can't be NULL.

Que $R = \{A_1, A_2, A_3 \dots A_n\}$ & $CK = \{A_1, A_2\}$ then superkey ?



$\Rightarrow sk(A_1) + sk(A_2) - sk(A_1 \cap A_2)$

$2^{n-1} + 2^{n-1} - 2^{n-2}$ ✓✓

② $CK = \{A_1A_2, A_1A_3\}$ then sk?

$sk(A_1A_2) + sk(A_1A_3) - sk(A_1A_2A_3)$

$\Rightarrow 2^{n-2} + 2^{n-2} - 2^{n-3}$ ✓

Que $\{A_1, A_2 - A_n\}$ then # CK?

$2^n - 1$ [ϕ can't be ck].

Scanned by CamScanner

Database Design & Normalization

Functional Dependency:

→ Trivial : $X \to Y$ [$X \supseteq Y$] eg. $A \to A$ or $AB \to A$

→ Non-Trivial : $X \to Y$ then $X \cap Y = \phi$

→ Semi : $X \to Y$ then $X \not\supseteq Y$ & $X \cap Y \neq \phi$.

X determine Y or Y is functionaly dependent on A.

Imp. Note for Exam: With instance we can't determine that a particular dependency will hold ~~or not~~. But we can conclude a particular dependency doesn't hold.

Inference Rules:

→ $A \to B$ if $A \supseteq B$

→ if $A \to B$ & $B \to C \Rightarrow A \to C$

→ if $A \to BC$ then $A \to B$ & $A \to C$

→ if $A \to B$ then $AC \to BC$

→ if $A \to BC$ then $A \to B$ & $A \to C$

→ if $A \to B$ & $A \to C$ then $A \to BC$

→ if $A \to B$ & $C \to D$ then $AC \to BD$

Equivalence of FD.

F: & G: and check $F \supseteq G$ & $G \supseteq F$.

Imp. Minimal Cover:

→ Split FD's. RHS should have single attribute. $A \to BC \Rightarrow A \to B$ & $A \to C$

→ find the redudant FDs & Remove [eg. $A \to B, B \to C, A \to C \Rightarrow A \to B, B \to C$]

→ find the redundat attribute on LHS & delete.

eg. $AB \to C$ ['A' can be removed if B^+ contains A]

Scanned by CamScanner

Multi Valued Dependency.

Note: Every FD is a special case of MD.

Normalization:

Sup: With 2-attributes table, relations follow all Normalization.

→ **Without Normalization:**
- Redundancy
- Anomalies → Insert
  - Update [At one place, left another]
  - Delete [only want Dep info not Emp].

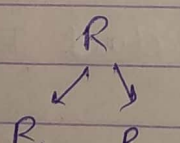
→ **Lossy Decomp:** $S = r_1 * r_2$

If lossy join then S will contain more tuples.

→ **Lossless join:**

↓

Common attribute should be key in atleast one Relation.



**

$(R_1 \cap R_2) \rightarrow R_1$ or $(R_1 \cap R_2) \rightarrow R_2$

or

$(R_1 \cap R_2) \rightarrow R_1 - R_2$ or $(R_1 \cap R_2) \rightarrow R_2 - R_1$

→ **functional Preserving:**

↓

More practice

Scanned by CamScanner

→ 1 NF : Atomic attribute [Neither multivalued nor Composite]

→ 2 NF :

↳ No partial dependency

[partial : $X \rightarrow Y$
(X ↓ part of key, Y ↓ Non prime attribute]

↳ If key has single attribute then No partial dependency.

→ 3 NF :

↳ No transitive dependency. (one non-prime determines other non-primes)

$X \rightarrow Y$
↓
either X is super key or Y is prime attribute.

↳ If all are prime attribute [part of any ck], then that is already in 3NF.

→ BCNF :

↳ $X \rightarrow Y$
↓
It should always be super key.

↳ Lossless decomposition always guarantee but FD preservation not guarantee.

Scanned by CamScanner

Multivalued dependency Rules:

- Complementation Rule: if $\alpha \twoheadrightarrow \beta$ holds on R then $\alpha \twoheadrightarrow [R-(\alpha \cup \beta)]$
- Multivalued Augmentation: $\alpha \twoheadrightarrow \beta$ and $\gamma \supseteq \delta \Rightarrow \alpha\gamma \twoheadrightarrow \beta\delta$
- Multivalued transitivity: $\alpha \twoheadrightarrow \beta$ and $\beta \twoheadrightarrow \gamma \Rightarrow \alpha \twoheadrightarrow \gamma$
- Coalescence Rule: if $\alpha \twoheadrightarrow \beta$ and there exist γ such that
  $\gamma \cap \beta = \phi$ and
  $\gamma \rightarrow \delta$ and $\beta \supseteq \delta$ then
  ~~$\alpha \rightarrow \beta$~~. $\alpha \rightarrow \delta$.

Imp: ~~Note~~
- In SQL: if <a, b> both are primary key then FK should have reference for both (a, b).

Scanned by CamScanner

SQL: → UNION, EXCEPT, INTERSECT ⇒ Removes duplicates
→ UNION ALL, EXCEPT ALL, INTERSECT ALL ⇒ Not Removes duplicates

Notes & Imp points:

→ An SQL query can contain a HAVING clause only if it has a GROUP BY clause.

→ All attributes used in the GROUP BY clause must appear in the SELECT clause.

Imp. → Every NULL is considered as separate group.

→ If you are using aggregate fun" with other attribute in SELECT then it is necessary to use GROUP BY.

eg.

Select A, Count(*)
FROM R
GROUP BY A
True / Right

Select A, Count(α)
FROM R
Wrong.

→ If you are using ORDER BY (by default value is Ascending) then that attribute should always be present in SELECT.

Imp. Aggregate fun": NULL values are removed / Ignored.
↳ Agg. fun" can used only in SELECT state".

eg. Salary + HR

1	5	— 6	⇒ Count = 2.
2	6	— 8	⇒ Count(salary) = 3
3	N	— N	⇒ Sum(salary) = 6
N	1	— N	⇒ Avg(salary) = 6/3

⇒ Avg(distinct salary) = 6/3

Scanned by CamScanner

Relational Model : Relational Algebra & Tuple Calculus

→ Π : Projection (col^{m}) , → σ : Selection (Rows)

→ duplicate values already removed.
→ UNION ALL : don't remove duplicates.

Sup 1. $|R| \geq |\Pi_{<attrs>}(R)|$

1. $|\Pi_{attribute}(R)| < |R|$ ⇒ May be, if attribute is not super key, duplicates will remove.
2. $|\Pi_{attr}(R)| = |R|$ ⇒ If <attrs> is super key. All values will be unique. No duplicates.

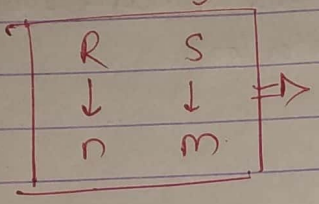
Scanned by CamScanner

Join:

→ Thetha : We don't compare any attribute with Constant in Condition.
<only one attribute with another attribute>

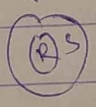
→ Natural : Compare common attribute.

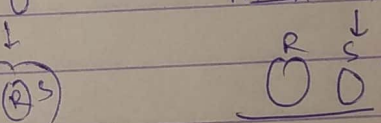
→ Outer joins (full, left, Right)



Need to update (video)

	Min	Max
X	mn	mn
⋈	0	
⟕	n	
⟖	m	
⟗	max(n, m)	
∪	Max(m,n)	m+n
∩	0	min(m,n)
−	0	~~max~~ m





Scanned by CamScanner

Division:

R

A	B
a_1	b_1
a_2	b_1
a_1	b_2

S

A
a_1
a_2

$\Rightarrow R \div S$ (for attribute $\uparrow$ $(R-S)$)

$\Rightarrow$

B
b_1

$T_1 \leftarrow \Pi_{(R-S)}(R)$

$T_2 \leftarrow \Pi_{(R-S)}((S \times T_1) - R)$

$T \leftarrow T_1 - T_2$

Que Nested loop Method, Join $r(R) < r(S)$. then Outer loop for ? To have fewer # disk block access.

GATE-2014

GATE overflow $\Rightarrow$ relation $r(R)$ is outer loop.

Ans

Scanned by CamScanner

Transaction Management & Concurrency Control

→ ACID - Atomicity, Consistency, Integrity, Durability.

✓ Schedule: T_1 T_2 ... T_m
n_1 n_2

$$\#\text{schedule} = \frac{(n_1 + n_2 + n_3 + \dots)!}{n_1!\, n_2! \dots}$$

$$\#\text{Serial schedule} = m!$$

✓ → Complete schedule: Commit or abort at the end of the schedule.

→ Recoverable: Commit check [for R-W, W-R].

→ Cascadeless: only Read after Commit, write can be performed before Comit.

→ strict: Read & Write both after Comit.

✓ Equivalent schedule:

① Result Equivalent

② Conflict Equivalent → Conflict opr^{n}s.
[same data, diff trans^n &
→RW, →WR →WW].

Scanned by CamScanner

✓ View Serializability:
- → Initial Read → if from DB
- → final write
- → Conflict order

✓ Conflict serializability:
- → Make graph.

Schedule
↓
Dependency graph
- Cycle → Blind write
  - Write without any Read. → Not CS, Not VS & Not S.
  - Yes → Polygraph
    - Cycle → Not CS, Not VS & Not S.
    - No cycle → Not CS, ~~Not~~ VS & S
  - No → Not View Serializable
- No cycle → C.S, V.S & S

Scanned by CamScanner

Locking & Concurrency

① Lock Based : Shared (Read), Exclusive (Write)
- ↳ deadlock may occur
- ↳ Not always produce serializable

② 2 PL : : Growing & shrinking phase
- ↳ deadlock may occur
- ↳ Always serializable
- ↳ cascading rollback problem.

③ Strict 2PL :
- ↳ Write / Exclusive lock release after Commit
- ↳ No cascading rollback
- ↳ deadlock may occur.

④ Rigurous 2PL :
- ↳ Shared & Exculsive both release after commit
- ↳ No cascading rollback
- ↳ deadlock may occur.

⑤ Constructive 2PL :
- ↳ No deadlock
- ↳ Hold all lock before execution
- ↳ No cascade rollback

++

⑥ Graph Based Algo :
- ↳ No deadlock [Tree structure, No cycle]
- ↳ lock parent to reach required data lock
- ↳ Always serilizable
- ↳ Early unlock of locks, so Concurrency ↑.

Scanned by CamScanner

(7) Timestamp Based protocol:

↳ No deadlock

↳ starvation possible

✓ $TS(T_i) < TS(T_j) \Rightarrow T_i \rightarrow T_j \Rightarrow$ Serial schedule.

✓ eg.

T_1	T_2
R(A)	
	W(A)
W(A)	

and given $TS(T_i) < TS(T_j)$

⇓

Conflict action should be in order $T_i \rightarrow T_j$ ie. $T_1 \rightarrow T_2$.

⇓

But T_2 write is before T_1. So, Rejected & Rollback with New timestamp.

✓ Thomas write Rule:

In above case, Thomas accept (W-W) in any order.

(8) Wait-die scheme: → Non-preemptive technique

→ older will wait

→ deadlock prevention. → Rollback of older process.

T_{22}	T_{23}	T_{24}
5	10	15

→ if T_{22} req. which is held by T_{23} then T_{22} will wait.

→ if T_{24} req. which is held by T_{23} then T_{24} will die.

Scanned by CamScanner

(9) Wound – wait scheme : → deadlock prevention technique

↳ preemptive

↳ Younger ~~will get resource~~ Transaction allowed to wait.

↳ if T_{24} req.

Rollback of older transaction.

eg.

T_{22}	T_{23}	T_{24}
5	10	15.

↳ if T_{22} request a data item held by T_{23} then data item will preempted from T_{23} and T_{23} will be rollbacked.

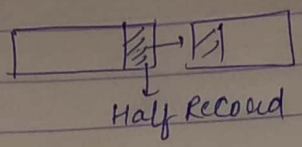
↳ if T_{24} is request to resource held by T_{23}, then T_{24} will wait.

Scanned by CamScanner

File Structure

→ Blocking factors : # Records per blocks

→ spanned Strategy :



⇒ No memory wastage.

Note: ✱✱

→ In Exam by default use non-spanned and use ceil. to find Blocking factor.

Don't directly do multiplication. ✱✱

→ Anchor Record : first Record of each block.

→ Ordered searching : $(\log_2 B)$ ⇒ Binary search ⇒ # Block Access Req.

→ Unordered : $(B/2)$ ⇒ Linear search ⇒ # Block Access Req.

✓ Indexing → Dense → # Entries = # Records

→ sparse → # Entries = # Blocks.

Index file always ordered.

Block Access $= (\log_2 B_i + 1)$

Index

Simple Level. → Multi-Level

B, B^+.

① Primary : → sparse

→ data order in primary key.

→ # Block Access $= \log_2 B_i + 1$

② Secondary : → Unordered, → dense.

③ clustered : → sparse + dense mix

→ data ordered on non-key

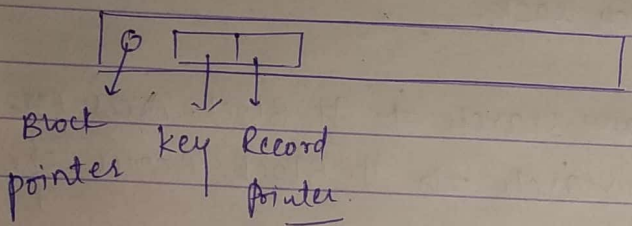
→ Best Base ⇒ $\log_2 B_i + 1$

Scanned by CamScanner

Multi-Level

B-Tree

- Root: 2 to 'p' children (p = order); 1 to (p-1) keys.
- Internal Node: $\lceil p/2 \rceil$ to p children.



- Leaf Node: $\lceil \frac{p-1}{2} \rceil$ to (p-1) keys or $\lceil p/2 \rceil$ to p children.
- O(log n): To Insert, delete & search

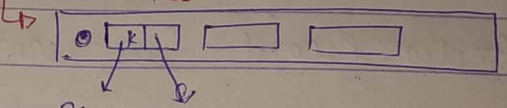
Ques: Suppose order = 23. How many records will be fill Level 4.

Level	# Nodes	Index Record	Disk pointer
1	1	22	23
2	23	23*22	23*23
3	23^2	23^2*22	23^3
4	23^3	23^3*22	23^4 Null

Add these

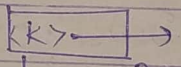
B^+ Tree

- Keys should be $\lceil p/2 \rceil$ to p. — (1)
- Internal



[Block, Key].

- Leafs:



key, Record + Next Block pointer

Ques. Order internal = ~~34~~ 23
Order Leaf = ~~31~~ 21
All nodes are ~~order~~ key entries

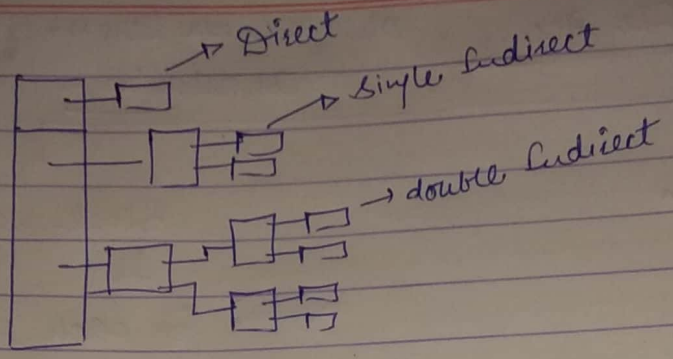
Level	# Nodes	# ~~Index~~	Disk
1	1	22	23
2	23	23X22	23X23
3	23^2	23^2X22	23^3
4	23^3	23^3X21	

Record Contains Index pointer.

Scanned by CamScanner

File System & Disk

✓ I-Node :



Que. Assuming 32-bit Address, with 1 KB Blocks and with 4-bytes per block pointers. Direct 10, Single 1, Double = 1, Triple = 1. What is the max file size possible?

$$\# \text{ pointers} = \frac{1\,KB}{4B} = 2^8 = 256.$$

With Direct : 10×256 KB

Single Indirect : 256×1 KB

double Indirect : $256 \times 256 \times 1$ KB

Triple Indirect : $256 \times 256 \times 256 \times 1$ KB

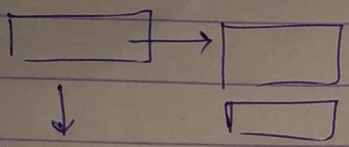
Add All. ++

Scanned by CamScanner

Ques. T_1 = 2000 Records T_2 = 400 Records
DB = 80 DB = 20

Joins with file Access : Always choose Less No of Records / DB as outer loop.

Nested loop join



for each Record, runner block Access.

=> 400 * 80 + 20 (A1)

Block Nested loop

for each record, the outer block access.

=> 20 * 80 + 20

Scanned by CamScanner